POO + AUTOMATIZACIÓN · LABORATORIO INTEGRADOR

Laboratorio #1

Soluciones de Negocio: POO + n8n

MODALIDAD	TEMAS	DURACIÓN SUGERIDA	NIVEL
Individual	**POO + n8n**	**1-2 semanas**	**Avanzado**

Nombre del estudiante: _____ **Fecha:** _____

Objetivo

Diseñar, construir y exponer una **solución de negocio completa**: un workflow funcional en n8n que automatice un proceso real (consumiendo JSON Server y al menos una API externa), aplicando **clases y objetos (POO) en JavaScript puro** para modelar la lógica de negocio que ese workflow ejecuta. Este laboratorio integra lo aprendido en Programación Orientada a Objetos y en Automatización con n8n — no son dos entregas separadas, son una sola solución, completamente en JavaScript, sin interfaz visual.

> **Importante:** Este trabajo es **INDIVIDUAL**. Puedes apoyarte en Inteligencia Artificial durante el desarrollo, pero debes poder explicar y defender cualquier parte de tu solución — el workflow, las clases, y cada decisión de diseño — durante la presentación final.

> **El estándar de este laboratorio:** no se trata de llenar un formulario ni de contestar preguntas — se trata de construir algo que **funcione de verdad**. Un workflow que nunca se ejecutó, o clases que nunca se probaron con datos reales, no cumplen el objetivo. Todo lo que entregues debe poder demostrarse en vivo.

Elige tu escenario de negocio

Selecciona **uno** de los siguientes 5 escenarios. Todos requieren JSON Server como una de tus fuentes de datos, combinado con al menos una API externa distinta. Puedes proponer una variación propia, siempre que mantenga el mismo nivel de complejidad y sea aprobada por el docente.

1. Central de facturación con días hábiles. Una empresa emite facturas en JSON Server y necesita calcular automáticamente su fecha de vencimiento (ej. 15 días hábiles después de emitida), sin caer en feriados. El workflow obtiene las facturas pendientes, consulta los feriados del año en la **Public Holiday API** (date.nager.at), calcula con un nodo `Code` la fecha real saltando feriados y fines de semana, y marca como "vencida" cualquier factura cuyo plazo ya pasó.

2. Pedidos con verificación geográfica de cobertura. Una tienda en línea solo despacha a ciertas zonas y calcula el costo de envío según la ubicación del cliente. El Webhook recibe el pedido (con la IP del cliente), el workflow consulta esa IP en la **IP-API** (geolocalización), decide con un `IF` si la zona está en cobertura, calcula el costo de envío, y actualiza el pedido en JSON Server — o lo rechaza con un motivo claro si está fuera de zona.

3. Recepción de cotizaciones por correo electrónico. Una empresa recibe solicitudes de cotización por correo, no por formulario web. Un `Email Trigger (IMAP)` detecta correos nuevos, un nodo `Code` extrae el producto y la cantidad solicitada del cuerpo del mensaje, el workflow consulta el stock en JSON Server, y registra la solicitud como "pendiente" o "sin stock" según corresponda. Este es el escenario que recibe información directamente del correo, sin pasar por un formulario.

4. Panel de riesgo de proveedores con comprobante QR. Antes de aprobar un pago, la empresa debe verificar que el proveedor no esté moroso ante Hacienda, y generar un comprobante visual del pago aprobado. El workflow consulta la **API de Hacienda** por cada proveedor con pagos pendientes en JSON Server; si no está moroso, genera un código **QR (api.qrserver.com)** con los datos del pago y lo asocia al registro; si está moroso, el pago queda retenido para revisión manual.

5. Monitor de tipo de cambio con ajuste automático de precios. Una empresa importadora necesita anticipar cuándo el tipo de cambio sube demasiado. Un `Schedule Trigger` diario consulta el **tipo de cambio del BCCR**, lo compara contra el histórico guardado en JSON Server con un nodo `Code`, y si la variación supera un porcentaje definido, recalcula y actualiza automáticamente los precios de los productos afectados.

Construye el workflow en n8n

El workflow debe ser completamente funcional, no solo diseñado en papel. Debe demostrar:

Componente	Mínimo	Propósito
Trigger	1	Webhook, Schedule o Email, según tu escenario.
HTTP Request	2	Uno hacia JSON Server, otro hacia tu API externa elegida.
IF o Filter	1	Al menos una bifurcación lógica real — no una secuencia lineal.
Code	1	Lógica de transformación o cálculo que las expresiones simples no resuelven.
Resultado final	1	Una salida verificable: un registro actualizado, una alerta, un comprobante.

El workflow debe contemplar al menos **dos situaciones de error** (ej. la API externa no responde, el dato esperado no llega, el producto o proveedor no existe).

Modela tu lógica de negocio con clases (POO)

Antes de escribir el nodo `Code` del workflow, diseña en JavaScript puro (sin ningún framework) las clases que representan las entidades de tu escenario. Requisitos mínimos:

- Al menos **2 clases** que modelen las entidades principales de tu escenario (ej. `Factura`, `Proveedor`, `Pedido`), cada una con al menos un método propio que implemente una regla de negocio real (ej. `calcularDiasRestantes()`, `estaEnRiesgo()`).
- Estas clases deben poder probarse de forma **independiente**: un archivo `main.js` que cree instancias con datos reales y muestre por consola el resultado de sus métodos, sin necesidad de que n8n esté corriendo.
- La lógica que implementaste en tus clases debe ser **la misma** que tu nodo `Code` aplica dentro del workflow sobre los datos reales — diseña primero con POO, y luego traduce esa lógica al nodo `Code`. No deben existir dos lógicas de negocio distintas por separado.

Reglas de negocio y manejo de errores

Define e implementa como mínimo **cinco reglas de negocio** relacionadas con tu escenario, y contempla al menos dos situaciones inesperadas (dato incompleto, servicio externo caído, información inexistente). Estas reglas deben ser visibles tanto en el comportamiento del workflow como en los métodos de tus clases.

Documento de lógica de negocio

Al finalizar, entrega un documento breve (2-4 páginas) que explique — con tus propias palabras, no como respuestas sueltas — lo siguiente:

- El problema elegido y por qué se beneficia de esta solución.
- El diseño del workflow, con un diagrama del recorrido de los datos.
- Las reglas de negocio implementadas y cómo se reflejan en el workflow y en tus clases.
- Cómo se conectan tus clases de JavaScript con el nodo Code de tu workflow, y qué lógica de negocio implementan.
- Las decisiones técnicas más relevantes que tomaste durante la construcción.

Presentación final

Debes exponer y defender tu solución en vivo. La presentación debe incluir:

1. El problema de negocio que elegiste y por qué importa.

2. El workflow funcionando en n8n, de principio a fin, con datos reales.

3. Las clases en JavaScript que diseñaste, qué responsabilidad tiene cada una, y cómo esa misma lógica vive dentro del nodo Code de tu workflow.

4. Al menos un caso de error manejado correctamente, demostrado en vivo.

5. Conclusiones: qué fue lo más difícil de integrar ambos temas.

Entregables

- Workflow de n8n funcionando — link a tu repositorio de Git con el JSON exportado.
- Código de tus clases en JavaScript (POO) — en el mismo repositorio de Git.
- Documento de lógica de negocio (PDF o Word).
- Presentación oral en vivo.